

实验4 组合与继承

说明

一、实验目的

二、实验准备

三、实验内容

1. 实验任务1

2. 实验任务2

3. 实验任务3

4. 实验任务4

四、实验结论

1. 实验任务1

2. 实验任务2

3. 实验任务3

4. 实验任务4

五、实验总结(选)

六、实验文档提交要求

七、博客园编辑器使用补充说明

说明

• 阅读文档

请阅读文档，明确各项任务的具体要求、提交方式与截止时间。

• 编程建议

随文档提供了验证性实验代码。请直接从压缩包解压使用。

设计性实验，请先独立思考、设计和编写。遇到问题或瓶颈时再用AI讨论/咨询。缺失思考和试错过程，会削弱训练效果。

• 提交规范

- 一次实验的所有任务写在一篇博客中
- 发布博客后，还需要在作业系统中提交链接
- 晚交的实验，按课程规则处理（分数折半）

一、实验目的

- 理解组合（has-a）：会用 C++ 写组合类，完成成员对象的构造、初始化与复用
- 理解继承（is-a）：会用 C++ 写单继承派生类，掌握公有继承、重写与多态
- 对比深化：通过实践对比，领悟组合与继承在设计思想、用法上的差异
- 面向问题：能根据对象关系选型，完成可扩展、易维护的类设计与实现

二、实验准备

系统浏览/复习以下教材章节：

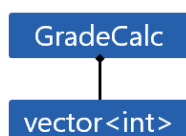
- 类的抽象与设计（第4-6章）
- 组合：解决的问题场景、定义和用法（第4章）
- 继承：解决的问题场景、定义和用法（第7-8章）

三、实验内容

1. 实验任务1

设计性实验任务：用**组合**实现成绩计算器类。运行、理解代码，回答问题。

- 问题场景描述



实现成绩计算器类 `GradeCalc`，采用**组合**方式内嵌 `vector<int>` 存放一门课程成绩，提供接口：

- 成绩录入
 - 成绩输出
 - 成绩排序（默认降序）
 - 最高分/最低分/平均分
 - 统计信息输出（包含分数段统计：[0, 60), [60, 70), [70, 80), [80, 90), [90, 100]）
- 代码组织
 - `GradeCalc.hpp` 类 `GradeCalc` 声明
 - `GradeCalc.cpp` 类 `GradeCalc` 实现
 - `demo1.cpp` 测试代码 + `main.cpp`

`GradeCalc.hpp`

```
1  #pragma once
2
3  #include <vector>
4  #include <array>
5  #include <string>
6
7  class GradeCalc {
8  public:
9      GradeCalc(const std::string &cname);
10     void input(int n);           // 录入n个成绩
11     void output() const;        // 输出成绩
12     void sort(bool ascending = false); // 排序（默认降序）
13     int min() const;            // 返回最低分（如成绩未录入，返回-1）
14     int max() const;            // 返回最高分（如成绩未录入，返回-1）
15     double average() const;     // 返回平均分（如成绩未录入，返回0.0）
16     void info();                // 输出课程成绩信息
17 }
```

```

18 private:
19     void compute();    // 成绩统计
20
21 private:
22     std::string course_name;    // 课程名
23     std::vector<int> grades;    // 课程成绩
24     std::array<int, 5> counts;    // 保存各分数段人数 ([0, 60), [60, 70), [70, 80), [80,
90), [90, 100]
25     std::array<double, 5> rates;    // 保存各分数段人数占比
26     bool is_dirty;    // 脏标记, 记录是否成绩信息有变更
27 };

```

GradeCalc.cpp

```

1  #include <algorithm>
2  #include <array>
3  #include <cstdlib>
4  #include <iomanip>
5  #include <iostream>
6  #include <numeric>
7  #include <string>
8  #include <vector>
9
10 #include "GradeCalc.hpp"
11
12 GradeCalc::GradeCalc(const std::string &cname):course_name{cname},is_dirty{true} {
13     counts.fill(0);
14     rates.fill(0);
15 }
16
17 void GradeCalc::input(int n) {
18     if(n < 0) {
19         std::cerr << "无效输入! 人数不能为负数\n";
20         std::exit(1);
21     }
22
23     grades.reserve(n);
24
25     int grade;
26
27     for(int i = 0; i < n; i++) {
28         std::cin >> grade;
29
30         if(grade < 0 || grade > 100) {
31             std::cerr << "无效输入! 分数须在[0,100]\n";
32             continue;
33         }
34
35         grades.push_back(grade);
36         ++i;
37     }
38
39     is_dirty = true;    // 设置脏标记: 成绩信息有变更

```

[illegible]

```

92         "[90, 100]"};
93
94     for(int i = grade_range.size()-1; i >= 0; --i)
95         std::cout << grade_range[i] << "\t: " << counts[i] << "人\t"
96         << std::fixed << std::setprecision(2) << rates[i]*100 << "%\n";
97 }
98
99 void GradeCalc::compute() {
100     if(grades.empty())
101         return;
102
103     counts.fill(0);
104     rates.fill(0.0);
105
106     // 统计各分数段人数
107     for(auto grade:grades) {
108         if(grade < 60)
109             ++counts[0];           // [0, 60)
110         else if (grade < 70)
111             ++counts[1];           // [60, 70)
112         else if (grade < 80)
113             ++counts[2];           // [70, 80)
114         else if (grade < 90)
115             ++counts[3];           // [80, 90)
116         else
117             ++counts[4];           // [90, 100]
118     }
119
120     // 统计各分数段比例
121     for(int i = 0; i < rates.size(); ++i)
122         rates[i] = counts[i] * 1.0 / grades.size();
123
124     is_dirty = false; // 更新脏标记
125 }

```

demo1.cpp

```

1  #include <iostream>
2  #include <string>
3  #include "GradeCalc.hpp"
4
5  void test() {
6      GradeCalc c1("OOP");
7
8      std::cout << "录入成绩:\n";
9      c1.input(5);
10
11     std::cout << "输出成绩:\n";
12     c1.output();
13
14     std::cout << "排序后成绩:\n";
15     c1.sort(); c1.output();
16

```

```

17     std::cout << "*****成绩统计信息*****\n";
18     c1.info();
19
20 }
21
22 int main() {
23     test();
24 }

```

编译、运行程序，录入成绩，结合运行结果，理解代码并回答问题。

问题1：组合关系识别

`GradeCalc` 类声明中，**逐行写出**所有体现"组合"关系的成员声明，并用一句话说明每个被组合对象的功能。

问题2：接口暴露理解

如在 `test` 模块中这样使用，是否合法？如不合法，解释原因。

```

1  GradeCalc c("OOP");
2  c.inupt(5);
3  c.push_back(97); // 合法吗？

```

问题3：架构设计分析

当前设计方案中，`compute` 在 `info` 模块中调用：

- (1) 连续打印3次统计信息，`compute` 会被调用几次？标记 `is_dirty` 起到什么作用？
- (2) 如新增 `update_grade(index, new_grade)`，这种设计需要更改 `compute` 调用位置吗？简洁说明理由。

问题4：功能扩展设计

要增加"中位数"统计，**不新增数据成员**怎么做？在哪个函数里加？写出伪代码。

问题5：数据状态管理

`GradeCalc` 和 `compute` 中都包含代码：`counts.fill(0); rates.fill(0);`。

`compute` 中能否去掉这两行？如去掉，在何种使用场景下会引发统计错误？

问题6：内存管理理解

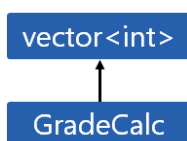
`input` 模块中代码 `grades.reserve(n)`；如果去掉：

- (1) 对程序功能有影响吗？（去掉重新编译、运行，观察功能是否受影响）
- (2) 对性能有影响吗？如有影响，用一句话陈述具体影响。

2. 实验任务2

设计性实验任务：用**继承**实现成绩计算器类。运行、理解代码，回答问题。

- 问题场景描述



实现成绩计算器类 `GradeCalc`，采用**继承**方式基于 `vector<int>` 创建，提供接口：

- 成绩录入

- 成绩输出
 - 成绩排序（默认降序）
 - 最高分/最低分/平均分
 - 统计信息输出（包含分数段统计：[0, 60), [60, 70), [70, 80), [80, 90), [90, 100])
- 代码组织
 - GradeCalc.hpp 类 GradeCalc 声明
 - GradeCalc.cpp 类 GradeCalc 实现
 - demo2.cpp 测试代码 + main.cpp

GradeCalc.hpp

```

1  #pragma once
2
3  #include <array>
4  #include <string>
5  #include <vector>
6
7  class GradeCalc: private std::vector<int> {
8  public:
9      GradeCalc(const std::string &cname);
10     void input(int n);           // 录入n个成绩
11     void output() const;        // 输出成绩
12     void sort(bool ascending = false); // 排序（默认降序）
13     int min() const;            // 返回最低分
14     int max() const;            // 返回最高分
15     double average() const;     // 返回平均分
16     void info();                // 输出成绩统计信息
17
18 private:
19     void compute();             // 计算成绩统计信息
20
21 private:
22     std::string course_name;    // 课程名
23     std::array<int, 5> counts;   // 保存各分数段人数 ([0, 60), [60, 70), [70, 80), [80,
24     90), [90, 100]
25     std::array<double, 5> rates; // 保存各分数段占比
26     bool is_dirty;              // 脏标记, 记录是否成绩信息有变更
27 };

```

GradeCalc.cpp

```

1  #include <algorithm>
2  #include <array>
3  #include <cstdlib>
4  #include <iomanip>
5  #include <iostream>
6  #include <numeric>
7  #include <string>
8  #include <vector>
9  #include "GradeCalc.hpp"

```

```

10
11
12 GradeCalc::GradeCalc(const std::string &cname): course_name{cname}, is_dirty{true}{
13     counts.fill(0);
14     rates.fill(0);
15 }
16
17 void GradeCalc::input(int n) {
18     if(n < 0) {
19         std::cerr << "无效输入! 人数不能为负数\n";
20         return;
21     }
22
23     this->reserve(n);
24
25     int grade;
26
27     for(int i = 0; i < n;) {
28         std::cin >> grade;
29         if(grade < 0 || grade > 100) {
30             std::cerr << "无效输入! 分数须在[0,100]\n";
31             continue;
32         }
33
34         this->push_back(grade);
35         ++i;
36     }
37
38     is_dirty = true;
39 }
40
41 void GradeCalc::output() const {
42     for(auto grade: *this)
43         std::cout << grade << ' ';
44     std::cout << std::endl;
45 }
46
47 void GradeCalc::sort(bool ascending) {
48     if(ascending)
49         std::sort(this->begin(), this->end());
50     else
51         std::sort(this->begin(), this->end(), std::greater<int>());
52 }
53
54 int GradeCalc::min() const {
55     if(this->empty())
56         return -1;
57
58     return *std::min_element(this->begin(), this->end());
59 }
60
61 int GradeCalc::max() const {
62     if(this->empty())

```



```

63         return -1;
64
65         return *std::max_element(this->begin(), this->end());
66     }
67
68     double GradeCalc::average() const {
69         if(this->empty())
70             return 0.0;
71
72         double avg = std::accumulate(this->begin(), this->end(), 0.0) / this->size();
73         return avg;
74     }
75
76     void GradeCalc::info() {
77         if(is_dirty)
78             compute();
79
80         std::cout << "课程名称:\t" << course_name << std::endl;
81         std::cout << "平均分:\t" << std::fixed << std::setprecision(2) << average() <<
std::endl;
82         std::cout << "最高分:\t" << max() << std::endl;
83         std::cout << "最低分:\t" << min() << std::endl;
84
85         const std::array<std::string, 5> grade_range{"[0, 60) ",
86                                                     "[60, 70)",
87                                                     "[70, 80)",
88                                                     "[80, 90)",
89                                                     "[90, 100]"};
90
91         for(int i = grade_range.size()-1; i >= 0; --i)
92             std::cout << grade_range[i] << "\t: " << counts[i] << "人\t"
93                 << std::fixed << std::setprecision(2) << rates[i]*100 << "%\n";
94     }
95
96     void GradeCalc::compute() {
97         if(this->empty())
98             return;
99
100         counts.fill(0);
101         rates.fill(0);
102
103         // 统计各分数段人数
104         for(int grade: *this) {
105             if(grade < 60)
106                 ++counts[0];           // [0, 60)
107             else if (grade < 70)
108                 ++counts[1];           // [60, 70)
109             else if (grade < 80)
110                 ++counts[2];           // [70, 80)
111             else if (grade < 90)
112                 ++counts[3];           // [80, 90)
113             else
114                 ++counts[4];           // [90, 100]

```

```

115     }
116
117     // 统计各分数段比例
118     for(int i = 0; i < rates.size(); ++i)
119         rates[i] = counts[i] * 1.0 / this->size();
120
121     is_dirty = false;
122 }

```

demo2.cpp

```

1  #include <iostream>
2  #include <string>
3  #include "GradeCalc.hpp"
4
5  void test() {
6      GradeCalc c1("OOP");
7
8      std::cout << "录入成绩:\n";
9      c1.input(5);
10
11     std::cout << "输出成绩:\n";
12     c1.output();
13
14     std::cout << "排序后成绩:\n";
15     c1.sort(); c1.output();
16
17     std::cout << "*****成绩统计信息*****\n";
18     c1.info();
19
20 }
21
22 int main() {
23     test();
24 }

```

编译、运行程序，录入成绩，结合运行结果，理解代码并回答问题。

问题1：继承关系识别

写出 `GradeCalc` 类声明体现"继承"关系的完整代码行。

问题2：接口暴露理解

当前继承方式下，基类 `vector<int>` 的接口会自动成为 `GradeCalc` 的接口吗？

如在 `test` 模块中这样用，能否编译通过？用一句话解释原因。

```

1  GradeCalc c("OOP");
2  c.input(5);
3  c.push_back(97); // 合法吗？

```

问题3：数据访问差异

对比继承方式与组合方式内部实现数据访问的一行典型代码。说明两种方式下的封装差异带来的数据访问接口差异。

```
1 // 组合方式
2 for(auto grade: grades) // 通过什么接口访问数据
3 // 略
4
5 // 继承方式
6 for(int grade: *this) // 通过什么接口访问数据
7 // 略
```

问题4: 组合 vs. 继承方案选择

你认为组合方案和继承方案，哪个更适合成绩计算这个问题场景？简洁陈述你的结论和理由。

3. 实验任务3

设计性实验任务：综合运用组合、继承、虚函数实现**用一个接口打印不同图形**。运行、理解代码，回答问题。

- 问题场景描述

综合运用组合、继承、虚函数，实现用一个接口打印各种图形。

- 代码组织

- Graph.hpp Graph类、Circle类、Rectangle类、Triangle类、Canvas类声明
- Graph.cpp 类实现
- demo3.cpp 测模块 + main

Graph.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <vector>
5
6 enum class GraphType {circle, triangle, rectangle};
7
8 // Graph类定义
9 class Graph {
10 public:
11     virtual void draw() {}
12     virtual ~Graph() = default;
13 };
14
15 // Circle类声明
16 class Circle : public Graph {
17 public:
18     void draw();
19 };
20
```

```

21 // Triangle类声明
22 class Triangle : public Graph {
23 public:
24     void draw();
25 };
26
27 // Rectangle类声明
28 class Rectangle : public Graph {
29 public:
30     void draw();
31 };
32
33 // Canvas类声明
34 class Canvas {
35 public:
36     void add(const std::string& type);    // 根据字符串添加图形
37     void paint() const;                  // 使用统一接口绘制所有图形
38     ~Canvas();                           // 手动释放资源
39
40 private:
41     std::vector<Graph*> graphs;
42 };
43
44 // 4. 工具函数
45 GraphType str_to_GraphType(const std::string& s); // 字符串转枚举类型
46 Graph* make_graph(const std::string& type); // 创建图形，返回堆对象指针

```

Graph.cpp

```

1  #include <algorithm>
2  #include <cctype>
3  #include <iostream>
4  #include <string>
5
6  #include "Graph.hpp"
7
8  // Circle类实现
9  void Circle::draw()    { std::cout << "draw a circle...\n"; }
10
11 // Triangle类实现
12 void Triangle::draw()  { std::cout << "draw a triangle...\n"; }
13
14 // Rectangle类实现
15 void Rectangle::draw() { std::cout << "draw a rectangle...\n"; }
16
17 // Canvas类实现
18 void Canvas::add(const std::string& type) {
19     Graph* g = make_graph(type);
20     if (g)
21         graphs.push_back(g);
22 }

```

```

23
24 void Canvas::paint() const {
25     for (Graph* g : graphs)
26         g->draw();
27 }
28
29 Canvas::~~Canvas() {
30     for (Graph* g : graphs)
31         delete g;
32 }
33
34 // 工具函数实现
35 // 字符串 → 枚举转换
36 GraphType str_to_GraphType(const std::string& s) {
37     std::string t = s;
38     std::transform(s.begin(), s.end(), t.begin(),
39         [](unsigned char c) { return std::tolower(c);});
40
41     if (t == "circle")
42         return GraphType::circle;
43
44     if (t == "triangle")
45         return GraphType::triangle;
46
47     if (t == "rectangle")
48         return GraphType::rectangle;
49
50     return GraphType::circle; // 缺省返回
51 }
52
53 // 创建图形，返回堆对象指针
54 Graph* make_graph(const std::string& type) {
55     switch (str_to_GraphType(type)) {
56     case GraphType::circle:    return new Circle;
57     case GraphType::triangle:  return new Triangle;
58     case GraphType::rectangle: return new Rectangle;
59     default: return nullptr;
60     }
61 }

```

demo3.cpp

```

1  #include <string>
2  #include "Graph.hpp"
3
4  void test() {
5      Canvas canvas;
6
7      canvas.add("circle");
8      canvas.add("triangle");
9      canvas.add("rectangle");
10     canvas.paint();
11 }

```

```
12
13 int main() {
14     test();
15 }
```

编译、运行程序。结合运行结果，理解代码并回答问题。

问题1：对象关系识别

- (1) 写出Graph.hpp中体现"组合"关系的成员声明代码行，并用一句话说明被组合对象的功能。
- (2) 写出Graph.hpp中体现"继承"关系的类声明代码行。

问题2：多态机制观察

- (1) Graph 中的 draw 若未声明成虚函数，Canvas::paint() 中 g->draw() 运行结果会有何不同？
- (2) 若 Canvas 类 std::vector<Graph*> 改成 std::vector<Graph>，会出现什么问题？
- (3) 若 ~Graph() 未声明成虚函数，会带来什么问题？

问题3：扩展性思考

若要新增星形 star，需在哪些文件做哪些改动？逐一列出。

问题4：资源管理

观察 make_graph 函数和 Canvas 析构函数：

- (1) make_graph 返回的对象在什么地方被释放？
- (2) 使用原始指针管理内存有何利弊？

4. 实验任务4

设计性实验：综合运用组合、继承、虚函数实现**用一个接口尝试所有玩具特异功能**。

具体要求如下：

- 设计毛绒玩具类 Toy
 - 数据成员：玩具名称、玩具类型等（更多数据成员请自行调研、扩充设计）
 - 接口：特异功能等（更多函数成员请自行调研、扩充设计）
- 设计玩具工厂类 ToyFactory，包含一组毛绒玩具。
 - 接口：显示工厂所有玩具信息（名称、类型、特异功能等）
- 编写测试模块、运行测试
- 代码组织
 - 类声明保存在xx.hpp, 类实现保存在xx.cpp, 测试模块和主体代码保存在demo4.cpp

说明*：

1. 本任务是设计性实验任务，题目只给出最小化、粗略描述，具体请调研市面上电子毛绒玩具并基于个人创造力做细化和拓展设计，包括：
 - (1) 方案确定（组合/继承）

- (2) 类的数据成员设计
 - (3) 函数成员设计 (接口和私有工具等)
2. 在实验结论中, 提供你设计这个应用的问题描述、对象关系、源码、测试截图

四、实验结论

1. 实验任务1

此部分书写内容:

- 分别给出GradeCalc.hpp, GradeCalc.cpp task1.cpp源码, 以及, 运行测试截图
- 回答问题

问题1: 组合关系识别

GradeCalc 类声明中, **逐行写出**所有体现"组合"关系的成员声明, 并用一句话说明每个被组合对象的功能。

问题2: 接口暴露理解

如在 test 模块中这样使用, 是否合法? 如不合法, 解释原因。

```
1 GradeCalc c("OOP");  
2 c.inupt(5);  
3 c.push_back(97); // 合法吗?
```

问题3: 架构设计分析

当前设计方案中, compute 在 info 模块中调用:

- (1) 连续打印3次统计信息, compute 会被调用几次? 标记 is_dirty 起到什么作用?
- (2) 如新增 update_grade(index, new_grade), 这种设计需要更改 compute 调用位置吗? 简洁说明理由。

问题4: 功能扩展设计

要增加"中位数"统计, **不新增数据成员**怎么做? 在哪个函数里加? 写出伪代码。

问题5: 数据状态管理

GradeCalc 和 compute 中都包含代码: counts.fill(0); rates.fill(0);。

compute 中能否去掉这两行? 如去掉, 在何种使用场景下会引发统计错误?

问题6: 内存管理理解

input 模块中代码 grades.reserve(n); 如果去掉:

- (1) 对程序功能有影响吗? (去掉重新编译、运行, 观察功能是否受影响)
- (2) 对性能有影响吗? 如有影响, 用一句话陈述具体影响。

2. 实验任务2

此部分书写内容:

- 分别给出GradeCalc.hpp, GradeCalc.cpp task2.cpp源码, 以及, 运行测试截图
- 回答问题

问题1：继承关系识别

写出 `GradeCalc` 类声明体现"继承"关系的完整代码行。

问题2：接口暴露理解

当前继承方式下，基类 `vector<int>` 的接口会自动成为 `GradeCalc` 的接口吗？

如在 `test` 模块中这样用，能否编译通过？用一句话解释原因。

```
1  GradeCalc c("OOP");
2  c.inupt(5);
3  c.push_back(97); // 合法吗？
```

问题3：数据访问差异

对比继承方式与组合方式内部实现数据访问的一行典型代码。说明两种方式下的封装差异带来的数据访问接口差异。

```
1  // 组合方式
2  for(auto grade: grades) // 通过什么接口访问数据
3  // 略
4
5  // 继承方式
6  for(int grade: *this) // 通过什么接口访问数据
7  // 略
```

问题4：组合 vs. 继承方案选择

你认为组合方案和继承方案，哪个更适合成绩计算这个问题场景？简洁陈述你的结论和理由。

3. 实验任务3

此部分书写内容：

- 分别给出 `Graph.hpp`, `Graph.cpp`, `task3.cpp` 源码，及，运行测试截图
- 回答问题

问题1：对象关系识别

(1) 写出 `Graph.hpp` 中体现"组合"关系的成员声明代码行，并用一句话说明被组合对象的功能。

(2) 写出 `Graph.hpp` 中体现"继承"关系的类声明代码行。

问题2：多态机制观察

(1) `Graph` 中的 `draw` 若未声明成虚函数，`Canvas::paint()` 中 `g->draw()` 运行结果会有何不同？

(2) 若 `Canvas` 类 `std::vector<Graph*>` 改成 `std::vector<Graph>`，会出现什么问题？

(3) 若 `~Graph()` 未声明成虚函数，会带来什么问题？

问题3：扩展性思考

若要新增星形 `Star`，需在哪些文件做哪些改动？逐一列出。

问题4：资源管理

观察 `make_graph` 函数和 `Canvas` 析构函数：

- (1) `make_graph` 返回的对象在什么地方被释放?
- (2) 使用原始指针管理内存有何利弊?

4. 实验任务4

此部分书写内容:

- 你设计的应用的问题场景描述
- 陈述各类之间的关系 (继承、组合等) 及设计理由
- 源码: `xx.hpp`, `xx.cpp`, `demo4.cpp`源码
- 运行测试截图

五、实验总结(选)

此部分书写内容: (可以包括但不限于以下内容)

- 提炼本次实验的核心收获与思考
- 记录新发现、真实感受与待解疑问
- 任意想补充的反馈或分享

六、实验文档提交要求

- **提交形式**

请将实验文档以**在线技术博客**形式提交, 发布后请在**相应实验作业**页面提交链接。

博客开通与提交方法详见: 「[博客园博客开通及发布步骤_2024级_OOP.pdf](#)」

- **撰写内容**

必写: 「四、实验结论」

选写: 「五、实验总结」

要求: 内容完整, 表述简练, 逻辑清晰, 图文整洁

- **截止时间**

通常为实验课后一周内, 具体**截止时间**以博客园发布的作业通知为准。 (**提交时间**以博客园文章页面显示的日期为准)

截止后, 欢迎通过[教学班级博客主页](#)相互评阅。

代码正确、风格规范、图文清晰、总结到位的博客, 欢迎点赞推荐, 一起进步!

七、博客园编辑器使用补充说明

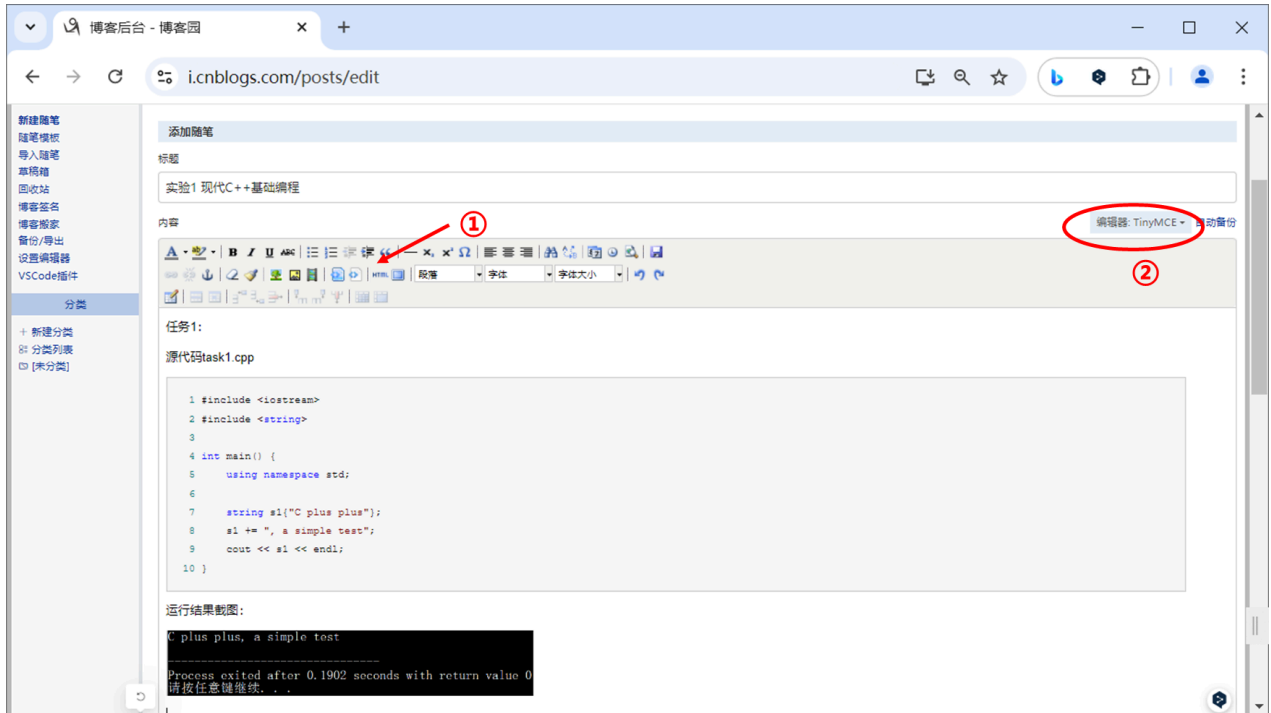
针对博客园平台使用的默认编辑器, 插入代码和图片后, 文字难以插入编辑的状况:

- 解决方案1

在插入代码和图片前，自己先编辑好文字，或者，用回车，预留好空白行

- 解决方案2

在编辑界面，点开页面对应的html源代码，在你想插入文字的地方加上html标签 `<p></p>`（图中①位置），然后再回到编辑界面，就会看到空白行。可以在那里编辑。



另，博客园平台设置页面支持更改默认编辑器（图中②位置）。目前，默认编辑器是markdown编辑器。

如果你暂不熟悉markdown基础用法，可切换成TinyMCE编辑器，其界面和使用风格接近office系列。

markdown编辑器使用参考： <https://www.runoob.com/markdown/md-tutorial.html>